

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Veritabanı Yedekleme ve Otomasyon

Enes Yıldız

22290180

github.com/enesoglu/blm4522

10.06.2026

İÇİNDEKİLER

1. Giriş	3
2. Yedekleme Otomasyon Mimarisi	4
3. Yedekleme Script'i	6
4. Zamanlama (Cron)	9
5. Loglama ve İzleme	10
6. Uyarı Mekanizmaları	12
7. Retention Politikası	14
8. Otomatik Yedek Doğrulama	16
9. Bulgular ve Sonuçlar	19

1. GİRİŞ

1.1 Proje Amacı

Bu teknik rapor, BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında PostgreSQL 18 üzerinde çalışan dvdrental veritabanı için yedekleme otomasyonunun tasarımını ve uygulanmasını incelemektedir. Çalışmanın temel odağı yedekleme türlerinin teorik karşılaştırması değil; betik geliştirme, zamanlama, loglama, izleme, uyarı üretme ve doğrulama adımlarının birlikte çalışan bir otomasyon hattı haline getirilmesidir.

Manuel yedek alma yaklaşımı, özellikle dağıtık ve servis sürekliliği beklenen sistemlerde yüksek operasyonel risk taşır. İnsan müdahalesine bağlı gecikmeler, unutulmuş komutlar, standart dışı dosya adları, kontrol edilmeyen disk tüketimi ve doğrulanmamış yedek dosyaları, felaket anında geri dönüş süresini belirsiz hale getirir. Bu nedenle yedekleme otomasyonu; yalnızca dosya üretmekten ibaret olmayan, ölçülebilir ve denetlenebilir bir işletim süreci olarak ele alınmalıdır.

- PostgreSQL 18 ve dvdrental veritabanı için yorumlu bir Bash yedekleme script'i geliştirmek.
- Linux cron zamanlayıcısı (scheduler) ile haftalık, günlük ve saatlik görevler tanımlamak.
- Yedekleme olaylarını zaman damgası, süre, dosya boyutu ve durum alanlarıyla loglamak.
- Saklama politikası (retention policy) ile eski yedekleri otomatik temizlemek.
- Hata oluştuğunda e-posta ve webhook tabanlı uyarı (alerting) mekanizmasını tetiklemek.
- Yedek bütünlüğünü pg_restore --list ve geçici veritabanına restore işlemiyle otomatik doğrulamak.

1.2 Metodoloji

Çalışmanın metodolojisi Script -> Schedule -> Monitor -> Alert -> Verify biçimindeki beş aşamalı otomasyon yaklaşımı üzerine kurulmuştur. Script aşamasında yedekleme komutları standartlaştırılır; Schedule aşamasında cron ile tekrar edilebilir hale getirilir; Monitor aşamasında log ve envanter verileri toplanır; Alert aşamasında eşik aşımı veya hata durumunda yöneticiye bildirim gönderilir; Verify aşamasında ise yedeğin gerçekten geri yüklenebilir olduğu otomatik olarak sınanır.

Tablo 1. Beş aşamalı otomasyon metodolojisi

Aşama	Amaç	Ölçülebilir çıktı
Script	pg_dump ve yardımcı kontrolleri standart betik içine almak	backup.sh, verify.sh
Schedule	Tekrarlı işleri cron ile zamanlamak	crontab görevleri
Monitor	Log, süre ve dosya boyutu metriklerini izlemek	backup_run_log kayıtları
Alert	Başarısızlık veya eşik aşımında uyarı üretmek	mail/webhook bildirimi
Verify	Yedeğin restore edilebilir olduğunu kanıtlamak	verify_status ve satır sayısı karşılaştırması

2. YEDEKLEME OTOMASYON MİMARİSİ

2.1 Genel Mimari

Otomasyon mimarisi, Linux işletim sistemindeki cron servisini tetikleyici katman olarak kullanır. Cron belirlenen zamanda backup.sh betiğini çalıştırır; betik pg_dump ile PostgreSQL 18 üzerindeki dvdrental veritabanının mantıksal yedeğini (logical backup) üretir; işlem sonucu log dosyasına ve opsiyonel olarak izleme tablosuna yazılır; başarısızlık halinde alarm mekanizması devreye girer.

```
cron -> backup.sh -> pg_dump -> log dosyası -> alarm mekanizması
```

Bu yapı Docker üzerinde çalışan PostgreSQL servisine dışarıdan bağlanabilecek şekilde tasarlanmıştır. Betikler konteyner içinde çalıştırılabileceği gibi, host Linux üzerinde PostgreSQL istemci araçları kurulu olduğu sürece konteyner portu üzerinden de çalıştırılabilir. Böylece otomasyon mantığı veritabanı motorundan ayrılır ve işletim sistemi seviyesinde yönetilebilir hale gelir.

2.2 Bileşenler Tablosu

Tablo 2. Yedekleme otomasyonu bileşenleri

Bileşen	Görev	Kullanılan araç
Script	Yedek alma, hata kontrolü, retention ve uyarı çağrılarını yürütür	backup.sh, verify.sh
Scheduler	Betikleri belirli zamanlarda otomatik çalıştırır	cron, crontab
Logger	Başlangıç, bitiş, süre, dosya boyutu ve durum bilgisini kaydeder	log dosyası, psql
Alerter	Kritik durumlarda DBA/DevOps ekibine bildirim gönderir	mail, sendmail, curl webhook
Verifier	Yedek dosyasını listeler ve geçici veritabanına geri yükler	pg_restore, createdb, dropdb

2.3 PostgreSQL Agent Benzeri Mantık

SQL Server ortamlarında zamanlanmış görevler çoğunlukla SQL Server Agent ile yönetilir. PostgreSQL tarafında aynı işlev tek bir merkezi agent bileşeni yerine Linux cron, Bash ve PostgreSQL istemci araçlarının birlikte kullanılmasıyla kurulur. Bu yaklaşım daha yalındır; ancak loglama, hata yakalama ve bildirim üretme gibi sorumlulukların script içinde açıkça tanımlanmasını gerektirir.

Bu projede cron yalnızca tetikleyici rolündedir. İş kuralı backup.sh içinde tutulur; pg_dump çıktısı standart dosya adlandırma şemasına göre saklanır; SQL tabanlı denetim kayıtları ise ops şeması altında tutulur. Böylece otomasyon katmanı hem işletim sistemi loglarıyla hem de veritabanı içi raporlarla izlenebilir.

SQL Kod Bloğu 1. Otomasyon izleme şeması

```
-- Yedekleme otomasyonu için izleme şeması
CREATE SCHEMA IF NOT EXISTS ops;

CREATE TABLE IF NOT EXISTS ops.backup_run_log (
  run_id      bigserial PRIMARY KEY,
  db_name     text NOT NULL,
  backup_level text NOT NULL CHECK (backup_level IN ('full','daily','hourly','verify')),
```

```

    backup_file    text,
    started_at     timestampz NOT NULL,
    finished_at    timestampz,
    duration_sec   numeric(10,2),
    file_size_mb   numeric(12,2),
    status         text NOT NULL CHECK (status IN ('SUCCESS','FAILED','WARNING')),
    message        text
);

CREATE TABLE IF NOT EXISTS ops.backup_alert_log (
    alert_id       bigserial PRIMARY KEY,
    run_id         bigint REFERENCES ops.backup_run_log(run_id),
    severity       text NOT NULL CHECK (severity IN ('INFO','WARNING','CRITICAL')),
    channel        text NOT NULL,
    alert_message  text NOT NULL,
    sent_at       timestampz DEFAULT now()
);

CREATE TABLE IF NOT EXISTS ops.backup_verify_log (
    verify_id      bigserial PRIMARY KEY,
    run_id         bigint REFERENCES ops.backup_run_log(run_id),
    backup_file    text NOT NULL,
    restore_db     text NOT NULL,
    expected_rows  bigint,
    restored_rows  bigint,
    verify_status  text NOT NULL CHECK (verify_status IN ('SUCCESS','FAILED')),
    verified_at    timestampz DEFAULT now()
);

```

3. YEDEKLEME SCRIPT'İ

3.1 backup.sh Script'inin Tam İçeriği

backup.sh betiği, yedekleme otomasyonunun merkezidir. Betik pg_dump komutunu çalıştırmakla kalmaz; izin hazırlama, dosya adı üretme, exit kodu kontrolü, dosya boyutu doğrulaması, retention temizliği, log yazımı ve alarm tetikleme adımlarını da aynı işlem akışı içinde yürütür.

Bash Kod Bloğu 1. Yorumlu backup.sh script'i

```

param(
    [string]$DbName = "dvdrental",
    [string]$HostName = "localhost",
    [int]$Port = 55432,
    [string]$UserName = "postgres",
    [string]$BackupLevel = "daily",
    [int]$RetentionDays = 7,
    [int]$MinSizeBytes = 1024,
    [string]$Root = "C:\Users\yild\projects_db\blm4522\artifacts\proje7"
)

$ErrorActionPreference = "Stop"

$pgBin = "C:\Program Files\PostgreSQL\18\bin"
$env:Path = "$pgBin;$env:Path"

$backupDir = Join-Path $Root "backups"
$logDir = Join-Path $Root "logs"
$alertDir = Join-Path $Root "alerts"
$logFile = Join-Path $logDir "pg_backup.log"

New-Item -ItemType Directory -Force -Path $backupDir, $logDir, $alertDir | Out-Null

$startedAt = Get-Date
$stamp = $startedAt.ToString("yyyyMMdd_HH:mm:ss")
$backupFile = Join-Path $backupDir "$DbName`_$BackupLevel`_$stamp.dump"

function Write-BackupLog {
    param([string]$Status, [string]$Message)
    $line = "{0} {1} | db={2} | level={3} | file={4} | {5}" -f `
        (Get-Date).ToString("yyyy-MM-dd HH:mm:ss"), $Status, $DbName, $BackupLevel, $backupFile, $Message
    Add-Content -Path $logFile -Value $line
    Write-Host $line
}

```

```

try {
    Write-BackupLog "START" "backup started"
    & pg_dump -U $UserName -h $HostName -p $Port -d $DbName -F c --no-owner --no-privileges -f $BackupFile
    $ExitCode = $LASTEXITCODE

    $Size = 0
    if (Test-Path $BackupFile) {
        $Size = (Get-Item $BackupFile).Length
    }

    if ($ExitCode -eq 0 -and $Size -gt $MinSizeBytes) {
        $Duration = [math]::Round(((Get-Date) - $StartedAt).TotalSeconds, 2)
        Write-BackupLog "OK" "size_bytes=$Size | duration_sec=$Duration"
    }
    else {
        throw "pg_dump failed or backup is too small. exit=$ExitCode size_bytes=$Size"
    }

    $Deleted = Get-Childitem $BackupDir -Filter "*.dump" |
        Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-$RetentionDays) }

    foreach ($file in $Deleted) {
        Remove-Item -LiteralPath $file.FullName -Force
        Write-BackupLog "RETENTION" "deleted_old_backup=$(($file.Name))"
    }
}
catch {
    $Message = $_.Exception.Message
    Write-BackupLog "FAIL" $Message

    $AlertFile = Join-Path $AlertDir "mail_$Stamp.txt"
    @"
TO: admin@example.com
SUBJECT: PG Backup FAIL
TIME: $(Get-Date -Format "yyyy-MM-dd HH:mm:ss")
DATABASE: $DbName
MESSAGE: $message

This file simulates an e-mail/webhook alert for the course demo.
"@ | Set-Content -Path $AlertFile -Encoding UTF8

    Write-Host "Alert file created: $AlertFile"
    exit 1
}

```

3.2 Script Parametreleri ve Değişkenleri

Tablo 3. backup.sh temel parametreleri

Parametre	Örnek değer	Açıklama
DB_NAME	dvdrental	Yedeklenecek PostgreSQL veritabanı adı
DB_USER	postgres	pg_dump ve psql işlemlerinde kullanılan kullanıcı
DB_HOST / DB_PORT	127.0.0.1 / 5432	Docker konteynerine veya yerel servise bağlantı noktası
BACKUP_DIR	/opt/pgbackups/dvdrental	Yedek dosyalarının saklanacağı dizin
LOG_DIR	/opt/pgbackups/logs	Script log dosyalarının yazılacağı dizin
RETENTION_DAYS	14	Otomatik temizleme için gün cinsinden saklama süresi
MIN_SIZE_BYTES	102400	Boş veya hatalı yedeği yakalamak için en küçük dosya boyutu
ALERT_EMAIL	dba@example.local	Kritik uyarıların gönderileceği e-posta adresi

3.3 Hata Kontrolü

Script iki temel hata kontrolü uygular. Birinci kontrol pg_dump komutunun çıkış kodudur. Unix/Linux ortamında \$? değeri 0 ise komut başarılı, 0 dışındaki değerler ise hatalı kabul edilir. İkinci kontrol, üretilen dosyanın boyutudur. Dosya beklenen minimum değerinin altındaysa yedekleme WARNING durumuyla işaretlenir ve uyarı gönderilir.

SQL Kod Bloğu 2. Başarısız yedek olayını kayıt altına alma

```
-- Script başarısızlık durumunda bu kayıt mantığıyla izleme tablosuna yazar
INSERT INTO ops.backup_run_log
(db_name, backup_level, backup_file, started_at, finished_at,
duration_sec, file_size_mb, status, message)
VALUES
('dvdrental', 'full', '/opt/pgbackups/dvdrental/dvdrental_full_2026-06-08.dump',
now() - interval '3 minutes', now(), 180, 0.00,
'FAILED', 'pg_dump başarısız oldu; PostgreSQL servisine ulaşılamadı.');
```

3.4 Script'in Elle Çalıştırma Çıktısı

Otomasyona alınmadan önce betik elle çalıştırılarak bağlantı, dosya üretimi, log yazımı ve alarm mekanizması doğrulanır. Aşağıdaki alan, gerçek terminal çıktısının rapora ekleneceği yerdir.

```
PS C:\Users\yildi\projects_db\blm4522> powershell.exe -ExecutionPolicy Bypass -File "$p7\backup.ps1" -BackupLevel full
[2026-06-09 20:07:48] START | db=dvdrental | level=full | file=C:\Users\yildi\projects_db\blm4522\artifacts\proje7\backups\dvdrental_full_2026-06-09-200748.dump | backup started
[2026-06-09 20:07:48] OK | db=dvdrental | level=full | file=C:\Users\yildi\projects_db\blm4522\artifacts\proje7\backups\dvdrental_full_2026-06-09-200748.dump | size_bytes=575234 | duration_sec=0.33
```

Tablo 4. Elle çalıştırma beklenen sonuç özeti

Kontrol	Beklenen sonuç
Komut	./backup.sh full
Exit code	0
Yedek dosyası	dvdrental_full_YYYY-MM-DD_HH-MM-SS.dump
Log dosyası	backup_dvdrental_YYYY-MM-DD_HH-MM-SS.log
İzleme kaydı	ops.backup_run_log.status = SUCCESS

4. ZAMANLAMA (CRON)

4.1 crontab Syntax Açıklaması

Cron, Linux sistemlerde komutların belirli zamanlarda otomatik çalıştırılması için kullanılan zamanlayıcıdır. Crontab satırları dakika, saat, ayın günü, ay ve haftanın günü alanlarından oluşur. Örneğin 0 2 * * * ifadesi her gün saat 02:00'de çalışmayı belirtir.

```
# dakika saat ayın_günü ay haftanın_günü komut
# 0 2 * * * /opt/pgbackups/bin/backup.sh full
```

4.2 Tam Yedek ve Günlük Artımsal Senaryo

PostgreSQL'de pg_dump komutu doğrudan fiziksel artımsal yedek üretmez. Bu nedenle bu çalışmada haftalık tam yedek pg_dump custom formatı ile alınırken, günlük görev değişim yoğun tablolar için data-only mantıksal döküm ve WAL arşivleme kontrolüyle desteklenen

artımsal senaryo olarak tanımlanmıştır. Amaç, teknik ayrıntıyı çoğaltmak değil, farklı yedek görevlerinin cron ile düzenli ve izlenebilir biçimde otomasyona bağlanmasını göstermektir.

Tablo 5. Zamanlama senaryosu

Görev	Sıklık	Komut	Amaç
Haftalık tam yedek	Pazar 02:00	backup.sh full	Tüm dvdrental veritabanının restore edilebilir ana kopyası
Günlük artımsal/diferansiyel mantık	Her gün 02:30	backup.sh daily	Değişim yoğun tablolar için daha küçük data-only döküm
Saatlik sağlık kontrolü	Her saat	verify.sh --latest-list-only	Son yedeğin pg_restore --list kontrolünü yapmak

4.3 crontab -e İçeriği

Cron Kod Bloğu 1. En az üç farklı zamanlama içeren crontab

```
# Haftalık tam yedek: Her pazar 02:00
0 2 * * 0 /opt/pgbackups/bin/backup.sh full >> /opt/pgbackups/logs/cron_full.log 2>&1

# Günlük artımsal/diferansiyel mantık: Her gün 02:30
30 2 * * * /opt/pgbackups/bin/backup.sh daily >> /opt/pgbackups/logs/cron_daily.log 2>&1

# Saatlik yedek liste/doğrulama kontrolü: Her saat başı
0 * * * * /opt/pgbackups/bin/verify.sh --latest-list-only >> /opt/pgbackups/logs/cron_verify.log 2>&1

# Aylık envanter raporu: Her ayın ilk günü 08:00
0 8 1 * * /opt/pgbackups/bin/inventory_report.sh >> /opt/pgbackups/logs/cron_inventory.log 2>&1
```

4.4 cron Loglarının Yazıldığı Yer (Windows Task Scheduler)

Cron çıktıları dağıtıma göre farklı yerlerde izlenebilir. Debian/Ubuntu sistemlerde /var/log/syslog içinde CRON kayıtları görülürken, RHEL/CentOS türevlerinde /var/log/cron dosyası kullanılır. systemd tabanlı dağıtımlarda journalctl -u cron veya journalctl -u crond komutları da kullanılabilir. Bu çalışmada her crontab satırı ayrıca uygulama log dosyasına yönlendirildiği için cron servis logu ile script logu birlikte değerlendirilebilir.

```
PS C:\Users\yildi\projects_db\blm4522> $action = "powershell.exe -ExecutionPolicy Bypass -File '$p7\backup.ps1' -BackupLevel daily"
PS C:\Users\yildi\projects_db\blm4522> schtasks /Create /TN "BLM4522_Proje7_DailyBackup" /TR $action /SC DAILY /ST 02:00 /F
SUCCESS: The scheduled task "BLM4522_Proje7_DailyBackup" has successfully been created.
PS C:\Users\yildi\projects_db\blm4522> schtasks /Query /TN "BLM4522_Proje7_DailyBackup" /V /FO LIST

Folder: \
HostName: DESKTOP-5QH5Q93
TaskName: \BLM4522_Proje7_DailyBackup
Next Run Time: 10.06.2026 02:00:00
Status: Ready
Logon Mode: Interactive only
Last Run Time: 30.11.1999 00:00:00
Last Result: 267011
Author: DESKTOP-5QH5Q93\yildi
Task To Run: powershell.exe -ExecutionPolicy Bypass -File C:\Users\yildi\projects_db\blm4522\artifacts\proje7\back
up.ps1 -BackupLevel daily
Start In: N/A
Comment: N/A
Scheduled Task State: Enabled
Idle Time: Disabled
Power Management: Stop On Battery Mode, No Start On Batteries
Run As User: yildi
Delete Task If Not Rescheduled: Disabled
Stop Task If Runs X Hours and X Mins: 72:00:00
Schedule: Scheduling data is not available in this format.
Schedule Type: Daily
Start Time: 02:00:00
Start Date: 9.06.2026
End Date: N/A
Days: Every 1 day(s)
```


5. LOGLAMA VE İZLEME

5.1 Log Dosyası Formatı

Loglama (logging), otomasyonun denetlenebilir olmasını sağlayan temel katmandır. Her log satırı ISO 8601 zaman damgası, seviye ve mesaj alanlarından oluşur. Buna ek olarak veritabanı içi ops.backup_run_log tablosu süre, dosya boyutu, durum ve açıklama alanlarıyla tarihsel analiz yapılmasını sağlar.

Tablo 6. Log dosyası alanları

Alan	Örnek	Açıklama
timestamp	2026-06-08T02:00:01+03:00	Olayın oluştuğu zaman
level	INFO / WARNING / ERROR	Operasyonel önem seviyesi
db_name	dvdrental	Yedeklenen veritabanı
backup_file	dvdrental_full_2026-06-08.dump	Üretilen dosya adı
duration_sec	186.40	Script çalışma süresi
file_size_mb	18.72	Yedek dosyasının MB cinsinden boyutu
status	SUCCESS	İşlem sonucu

5.2 Log Rotasyonu

Script logları sürekli büyüyebileceği için logrotate ile günlük rotasyon, sıkıştırma ve belirli sayıda eski dosya saklama uygulanır. Böylece log dosyalarının disk alanını kontrolsüz tüketmesi engellenir.

Linux Kod Bloğu 1. logrotate yapılandırması

```
/opt/pgbackups/logs/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
    create 0640 postgres postgres
    dateext
}
```

5.3 Tarihsel Log Analizi Sorgusu

SQL Kod Bloğu 3. Son 7 günde başarısız yedek sayısı

```
-- Son 7 gündeki başarısız veya uyarılı yedekleri sayma
SELECT
    status,
    COUNT(*) AS run_count,
    MIN(started_at) AS first_seen,
    MAX(started_at) AS last_seen
FROM ops.backup_run_log
WHERE started_at >= now() - interval '7 days'
    AND status IN ('FAILED', 'WARNING')
GROUP BY status
ORDER BY status;
```

5.4 Yedek Envanteri Raporu

Yedek envanteri raporu, hangi dosyanın ne zaman üretildiğini, ne kadar yer kapladığını ve son doğrulama durumunu gösterir. Bu rapor DBA ekibinin yalnızca log satırlarına bakmadan yedek zincirinin güncelliğini değerlendirmesini sağlar.

SQL Kod Bloğu 4. Yedek envanteri görünümü

```
-- Başarılı yedekleri en son doğrulama sonucu ile birlikte listeleme
CREATE OR REPLACE VIEW ops.v_backup_inventory AS
SELECT
  r.run_id,
  r.db_name,
  r.backup_level,
  r.backup_file,
  r.started_at,
  r.file_size_mb,
  r.status AS backup_status,
  v.verify_status,
  v.verified_at
FROM ops.backup_run_log r
LEFT JOIN LATERAL (
  SELECT verify_status, verified_at
  FROM ops.backup_verify_log v
  WHERE v.run_id = r.run_id
  ORDER BY verified_at DESC
  LIMIT 1
) v ON true
WHERE r.status = 'SUCCESS'
ORDER BY r.started_at DESC;

SELECT * FROM ops.v_backup_inventory LIMIT 10;
```

Tablo 7. Yedek envanteri örnek raporu

Seviye	Tarih	Boyut (MB)	Yedek durumu	Doğrulama
full	2026-06-07 02:00	18.72	SUCCESS	SUCCESS
daily	2026-06-08 02:30	4.18	SUCCESS	SUCCESS
hourly	2026-06-08 10:00	18.75	SUCCESS	LIST OK
daily	2026-06-06 02:30	4.09	SUCCESS	SUCCESS

6. UYARI MEKANİZMALARI

6.1 E-posta Uyarı Entegrasyonu

Uyarı mekanizması, otomasyonun başarısızlığı sessizce saklamasını engeller. backup.sh içinde mail veya sendmail komutu varsa hata mesajı DBA adresine gönderilir. Mail altyapısı yoksa script bunu WARNING olarak loglar; böylece uyarı kanalının kendisi de izlenebilir hale gelir.

Linux Kod Bloğu 2. E-posta uyarı komutu

```
printf "%s\n" "dvdrental yedekleme hatası oluştu. Log: ${LOG_FILE}" \
| mail -s "CRITICAL: PostgreSQL backup failed" dba@example.local
```

6.2 Hata Senaryosu Testi

Hata senaryosu testinde PostgreSQL konteyneri kasıtlı olarak durdurulur ve backup.sh tetiklenir. Beklenen sonuç pg_dump bağlantı hatası, FAILED durumunda ops.backup_run_log kaydı ve e-posta uyarısıdır. Test tamamlandıktan sonra konteyner yeniden başlatılır.

Linux Kod Bloğu 3. Kasıtlı hata oluşturma testi

```
# Kasıtlı hata: PostgreSQL konteynerini durdur
docker stop pg16-dvdrental
```

```
# Yedekleme script'ini tetikle
/opt/pgbackups/bin/backup.sh full
```

```
# Testten sonra servisi geri aç
docker start pg16-dvdrental
```

```
PS C:\Users\yildi\projects_db\blm4522> powershell.exe -ExecutionPolicy Bypass -File "$p7\backup.ps1" -Port 59999 -BackupLevel fail_test
[2026-06-09 20:10:21] START | db=dvdrental | level=fail_test | file=C:\Users\yildi\projects_db\blm4522\artifacts\proje7\backups\dvdrental_fa
il_test_20260609_201021.dump | backup started
pg_dump: data: connection to server at "localhost" (::1), port 59999 failed: Connection refused (0x0000274D/10061)
Is the server running on that host and accepting TCP/IP connections?
connection to server at "localhost" (127.0.0.1), port 59999 failed: Connection refused (0x0000274D/10061)
Is the server running on that host and accepting TCP/IP connections?
[2026-06-09 20:10:25] FAIL | db=dvdrental | level=fail_test | file=C:\Users\yildi\projects_db\blm4522\artifacts\proje7\backups\dvdrental_fa
il_test_20260609_201021.dump | pg_dump failed or backup is too small. exit=1 size_bytes=0
Alert file created: C:\Users\yildi\projects_db\blm4522\artifacts\proje7>alerts\mail_20260609_201021.txt
```

```
PS C:\Users\yildi\projects_db\blm4522> Get-Content (Get-ChildItem "$p7>alerts" | Sort-Object LastWriteTime -Descending | Select-Object -Fir
st 1).FullName
TO: admin@example.com
SUBJECT: PG Backup FAIL
TIME: 2026-06-09 20:10:25
DATABASE: dvdrental
MESSAGE: pg_dump failed or backup is too small. exit=1 size_bytes=0
```

6.3 Slack/Webhook Entegrasyonu

Operasyon ekipleri e-posta dışında Slack, Mattermost veya benzeri webhook kanallarını kullanabilir. Script içinde WEBHOOK_URL değişkeni doluysa curl komutu ile JSON mesajı gönderilir. Bu kanal özellikle mesai dışı kritik uyarıların görünürlüğünü artırır.

Linux Kod Bloğu 4. Webhook uyarı örneği

```
curl -fsS -X POST \
-H "Content-Type: application/json" \
--data '{"text":"CRITICAL: dvdrental backup failed. Check /opt/pgbackups/logs."}' \
"${WEBHOOK_URL}"
```

6.4 Uyarı Eşikleri Tablosu

Tablo 8. Uyarı eşikleri

Seviye	Koşul	Aksiyon
CRITICAL	pg_dump exit code 0 dışında veya PostgreSQL bağlantısı yok	E-posta + webhook + FAILED kayıt
WARNING	Yedek dosyası minimum boyutun altında	E-posta + WARNING kayıt
WARNING	Disk kullanımı %85 üzerinde	Webhook + günlük izleme raporu
INFO	Yedek başarıyla tamamlandı	Sadece log ve ops.backup_run_log kaydı
CRITICAL	Doğrulama restore işlemi başarısız	E-posta + webhook + doğrulama FAILED kaydı

SQL Kod Bloğu 5. Kritik uyarı olaylarını raporlama

```
-- Kritik uyarıların son durumunu izleme
SELECT
  a.severity,
  a.channel,
  a.alert_message,
  a.sent_at,
  r.backup_file
```

```
FROM ops.backup_alert_log a
JOIN ops.backup_run_log r ON r.run_id = a.run_id
WHERE a.severity = 'CRITICAL'
ORDER BY a.sent_at DESC
LIMIT 20;
```

7. RETENTION POLİTİKASI

7.1 Eski Yedeklerin Temizlenmesi

Retention politikası, yedek dosyalarının ne kadar süre saklanacağını ve hangi dosyaların otomatik silineceğini belirler. backup.sh içinde find -mtime +N mantığı kullanılarak belirlenen günden daha eski dosyalar temizlenir. Bu işlem log dosyasına yazıldığı için hangi dosyaların silindiği denetlenebilir.

Linux Kod Bloğu 5. find -mtime +N ile temizlik

```
find "/opt/pgbackups/dvdrental" -type f -name "dvdrental_*.dump" \
-mtime +14 -print -delete >> "/opt/pgbackups/logs/retention.log" 2>&1
```

7.2 GFS Politikası

Grandfather-Father-Son (GFS) politikası, yedekleri günlük, haftalık ve aylık katmanlara ayırır. Son yedi günlük yedekler hızlı geri dönüş için saklanır; haftalık yedekler kısa dönem operasyonel güvence sağlar; aylık yedekler ise daha uzun dönem arşiv niteliğindedir. Böylece her gün tüm eski dosyaları sınırsız saklamadan farklı zaman ölçeklerinde geri dönüş noktaları korunur.

7.3 Uygulamada GFS Örneği

Tablo 9. GFS saklama planı

Katman	Saklama süresi	Örnek dosya	Amaç
Son	7 günlük	dvdrental_daily_*.dump	Yakın tarihli operasyonel geri dönüş
Baba	4 haftalık	dvdrental_full_pazar_*.dump	Haftalık ana geri dönüş noktası
Büyükbaba	6 aylık	dvdrental_monthly_*.dump	Dönemsel arşiv ve denetim ihtiyacı

7.4 Disk Kullanımı İzleme

Disk kullanımı yalnızca dosya sistemi komutlarıyla değil, veritabanı içi büyüklük sorgularıyla da izlenmelidir. Böylece dvdrental veritabanının büyüme eğilimi ile yedek dosyası boyutları birlikte değerlendirilir.

SQL Kod Bloğu 6. Veritabanı boyutu ve son yedek boyutları

```
-- Veritabanı boyutu ile son yedek boyutlarını birlikte izleme
SELECT
  current_database() AS db_name,
  pg_size_pretty(pg_database_size(current_database())) AS database_size,
  (
    SELECT round(avg(file_size_mb), 2)
    FROM ops.backup_run_log
    WHERE started_at >= now() - interval '7 days'
    AND status = 'SUCCESS'
  ) AS avg_backup_size_mb,
  (
    SELECT max(file_size_mb)
    FROM ops.backup_run_log
    WHERE status = 'SUCCESS'
```

```
) AS max_backup_size_mb;
```

8. OTOMATİK YEDEK DOĞRULAMA

8.1 Doğrulama Script'i

Yedekleme dosyasının üretilmiş olması tek başına yeterli değildir. Otomatik doğrulama, dosyanın pg_restore tarafından okunabildiğini ve geçici bir veritabanına restore edilebildiğini kontrol eder. Bu adım felaket anından önce geri dönüş kabiliyetini kanıtladığı için otomasyonun en kritik halkasıdır.

Bash Kod Bloğu 2. verify.sh doğrulama script'i

```
#!/usr/bin/env bash
# PostgreSQL yedek doğrulama otomasyonu

set -u

DB_NAME="${DB_NAME:-dvdrental}"
DB_USER="${DB_USER:-postgres}"
DB_HOST="${DB_HOST:-127.0.0.1}"
DB_PORT="${DB_PORT:-5432}"
BACKUP_DIR="${BACKUP_DIR:-/opt/pgbackups/dvdrental}"
LOG_DIR="${LOG_DIR:-/opt/pgbackups/logs}"
MODE="${1:-restore-check}"
BACKUP_FILE="${2:-$(ls -lt ${BACKUP_DIR}/${DB_NAME} *.dump | head -n 1)}"
TEMP_DB="${DB_NAME}_verify_$(date +%Y%m%d%H%M%S)"
LOG_FILE="${LOG_DIR}/verify_${DB_NAME}_$(date +%Y-%m-%d_%H-%M-%S).log"

mkdir -p "${LOG_DIR}"

log() {
    echo "$(date --iso-8601=seconds) | $1 | $2" | tee -a "${LOG_FILE}"
}

if [[ -z "${BACKUP_FILE}" ]] || ! -f "${BACKUP_FILE}"; then
    log "ERROR" "Doğrulanacak yedek dosyası bulunamadı."
    exit 1
fi

log "INFO" "pg_restore --list kontrolü başlatıldı: ${BACKUP_FILE}"
pg_restore --list "${BACKUP_FILE}" > "${LOG_FILE}.list" 2>> "${LOG_FILE}"
LIST_EXIT=$?

if [[ ${LIST_EXIT} -ne 0 ]]; then
    log "ERROR" "pg_restore --list başarısız oldu."
    exit 2
fi

if [[ "${MODE}" == "--latest-list-only" ]]; then
    log "INFO" "Liste kontrolü başarılı; restore testi yapılmadı."
    exit 0
fi

dropdb --if-exists --host="${DB_HOST}" --port="${DB_PORT}" --username="${DB_USER}" \
    "${TEMP_DB}" >> "${LOG_FILE}" 2>&1
createdb --host="${DB_HOST}" --port="${DB_PORT}" --username="${DB_USER}" \
    "${TEMP_DB}" >> "${LOG_FILE}" 2>&1

pg_restore --host="${DB_HOST}" --port="${DB_PORT}" --username="${DB_USER}" \
    --dbname="${TEMP_DB}" --clean --if-exists \
    "${BACKUP_FILE}" >> "${LOG_FILE}" 2>&1
RESTORE_EXIT=$?

EXPECTED_ROWS="$(psql -At "host=${DB_HOST} port=${DB_PORT} user=${DB_USER} dbname=${DB_NAME}" \
    -c "SELECT COUNT(*) FROM public.rental;" 2>> "${LOG_FILE}")"
RESTORED_ROWS="$(psql -At "host=${DB_HOST} port=${DB_PORT} user=${DB_USER} dbname=${TEMP_DB}" \
    -c "SELECT COUNT(*) FROM public.rental;" 2>> "${LOG_FILE}")"

if [[ ${RESTORE_EXIT} -eq 0 && "${EXPECTED_ROWS}" == "${RESTORED_ROWS}" ]]; then
    VERIFY_STATUS="SUCCESS"
    log "INFO" "Restore doğrulaması başarılı. rental=${RESTORED_ROWS}"
else
    VERIFY_STATUS="FAILED"
    log "ERROR" "Restore doğrulaması başarısız. expected=${EXPECTED_ROWS}, restored=${RESTORED_ROWS}"
fi

psql "host=${DB_HOST} port=${DB_PORT} user=${DB_USER} dbname=${DB_NAME}" <<SQL >> "${LOG_FILE}" 2>&1
```

```

INSERT INTO ops.backup_verify_log
(backup_file, restore_db, expected_rows, restored_rows, verify_status)
VALUES
('${BACKUP_FILE}', '${TEMP_DB}', ${EXPECTED_ROWS:-0}, ${RESTORED_ROWS:-0}, ${VERIFY_STATUS});
SQL

dropdb --if-exists --host="${DB_HOST}" --port="${DB_PORT}" --username="${DB_USER}" \
"${TEMP_DB}" >>"${LOG_FILE}" 2>&1

[[ "${VERIFY_STATUS}" == "SUCCESS" ]] && exit 0 || exit 3

```

8.2 pg_restore --list ile Bütünlük Kontrolü

pg_restore --list komutu custom format yedek dosyasının içeriğini okur ve nesne listesini üretir. Bu komutun başarılı olması, dosyanın PostgreSQL restore aracı tarafından okunabilir olduğunu gösterir. Ancak yalnızca liste kontrolü yeterli değildir; bu nedenle geçici veritabanına restore testiyle desteklenir.

```

PS C:\Users\yildi\projects_db\blm4522> pg_restore --list $latest.FullName | Select-Object -First 20
;
; Archive created at 2026-06-09 20:13:30
; dbname: dvdrental
; TOC Entries: 151
; Compression: gzip
; Dump Version: 1.16-0
; Format: CUSTOM
; Integer: 4 bytes
; Offset: 8 bytes
; Dumped from database version: 18.3
; Dumped by pg_dump version: 18.3
;
; Selected TOC Entries:
;
897; 1247 16386 TYPE public mpaa_rating postgres
900; 1247 16398 DOMAIN public year postgres
256; 1255 16400 FUNCTION public _group_concat(text, text) postgres
257; 1255 16401 FUNCTION public film_in_stock(integer, integer) postgres
258; 1255 16402 FUNCTION public film_not_in_stock(integer, integer) postgres

```

8.3 Geçici DB'ye Geri Yükleme ve Satır Sayısı Karşılaştırması

SQL Kod Bloğu 7. Kaynak dvdrental satır sayısı özeti

```

-- Kaynak veritabanındaki kritik tabloların satır sayısı
SELECT 'film' AS table_name, COUNT(*) AS row_count FROM public.film
UNION ALL
SELECT 'customer', COUNT(*) FROM public.customer
UNION ALL
SELECT 'rental', COUNT(*) FROM public.rental
UNION ALL
SELECT 'payment', COUNT(*) FROM public.payment
ORDER BY table_name;

```

SQL Kod Bloğu 8. Restore edilen geçici veritabanı satır sayısı

```

-- Geçici restore veritabanında aynı tabloları sayma
SELECT 'film' AS table_name, COUNT(*) AS row_count FROM public.film
UNION ALL
SELECT 'customer', COUNT(*) FROM public.customer
UNION ALL
SELECT 'rental', COUNT(*) FROM public.rental
UNION ALL
SELECT 'payment', COUNT(*) FROM public.payment
ORDER BY table_name;

```

SQL Kod Bloğu 9. Kaynak ve restore satır sayısı karşılaştırması

```

-- Doğrulama sonucu tutarlı mı?
WITH source_counts AS (
  SELECT 'film' AS table_name, COUNT(*) AS row_count FROM public.film
  UNION ALL SELECT 'customer', COUNT(*) FROM public.customer
  UNION ALL SELECT 'rental', COUNT(*) FROM public.rental
  UNION ALL SELECT 'payment', COUNT(*) FROM public.payment
),
restore_counts AS (
  -- Bu sorgu geçici DB bağlantısında aynı mantıkla çalıştırılır ve sonuçlar karşılaştırılır.
  SELECT 'film' AS table_name, 1000::bigint AS row_count
  UNION ALL SELECT 'customer', 599

```

```

UNION ALL SELECT 'rental', 16044
UNION ALL SELECT 'payment', 14596
)
SELECT
s.table_name,
s.row_count AS source_rows,
r.row_count AS restored_rows,
CASE WHEN s.row_count = r.row_count THEN 'OK' ELSE 'MISMATCH' END AS compare_status
FROM source_counts s
JOIN restore_counts r USING (table_name)
ORDER BY s.table_name;

```

8.4 Doğrulama Sonuçlarının Ana Log'a Entegrasyonu

verify.sh sonucu ops.backup_verify_log tablosuna yazılır ve backup_run_log ile ilişkilendirilebilir. Bu sayede yedekleme başarılı görünse bile doğrulaması yapılmamış veya başarısız olmuş dosyalar envanter raporunda ayırt edilir.

Tablo 10. Otomatik doğrulama örnek sonucu

Tablo	Kaynak satır	Restore satır	Durum
customer	599	599	OK
rental	16044	16044	OK
payment	7660	7660	OK

```

PS C:\Users\yildi\projects_db\blm4522> powershell.exe -ExecutionPolicy Bypass -File "$p7\verify_backup.ps1"
[2026-06-09 20:15:42] VERIFY_START | db=dvdrental | backup=C:\Users\yildi\projects_db\blm4522\artifacts\proje7\backups\dvdrental_retention_
demo_20260609_201330.dump
Original database row counts:
table_name | count
-----
customer   | 599
payment    | 7660
rental     | 16044
(3 sat'r)

Restored database row counts:
table_name | count
-----
customer   | 599
payment    | 7660
rental     | 16044
(3 sat'r)

[2026-06-09 20:15:43] VERIFY_OK | db=dvdrental | restored_database=verify_20260609_201542 | backup=dvdrental_retention_demo_20260609_201330
.dump

```

9. BULGULAR VE SONUÇLAR

9.1 Otomasyon Öncesi/Sonrası Karşılaştırma

Otomasyon öncesinde yedekleme işlemi DBA tarafından elle komut çalıştırma, dosya kontrolü ve gerektiğinde manuel kopyalama adımlarına dayanır. Otomasyon sonrasında aynı süreç cron tarafından tetiklenen, loglanan, uyarı üreten ve restore testiyle doğrulanan bir iş akışına dönüşmüştür.

Tablo 11. Otomasyon öncesi/sonrası karşılaştırma

Metrik	Manuel süreç	Otomasyon sonrası
İnsan müdahale süresi	Her yedek için 10-15 dakika	Normal durumda 0 dakika
Hata sıklığı	Komut unutma veya yanlış izin riski yüksek	Standart script ile düşük

Metrik	Manuel süreç	Otomasyon sonrası
Kaçırılmış yedek sayısı	Takvim takibine bağlı	cron ve log ile izlenebilir
Dosya adı standardı	Operatöre bağlı	Tarih damgalı ve seviyeli adlandırma
Eski yedek temizliği	Manuel silme	RETENTION_DAYS ve GFS mantığı
Geri yüklenebilirlik kanıtı	Çoğunlukla varsayım	pg_restore ve geçici DB testi

9.2 Aylık Yedekleme İstatistiği Özet Tablosu

SQL Kod Bloğu 10. Aylık yedekleme istatistiği

```
-- Aylık başarı oranı, ortalama süre ve ortalama dosya boyutu
SELECT
  date_trunc('month', started_at)::date AS month_start,
  COUNT(*) AS total_runs,
  COUNT(*) FILTER (WHERE status = 'SUCCESS') AS success_runs,
  COUNT(*) FILTER (WHERE status <> 'SUCCESS') AS problem_runs,
  round(AVG(duration_sec), 2) AS avg_duration_sec,
  round(AVG(file_size_mb), 2) AS avg_file_size_mb,
  round(
    COUNT(*) FILTER (WHERE status = 'SUCCESS') * 100.0 / NULLIF(COUNT(*), 0),
    2
  ) AS success_ratio_percent
FROM ops.backup_run_log
GROUP BY date_trunc('month', started_at)
ORDER BY month_start DESC;
```

Tablo 12. Aylık yedekleme istatistiği örneği

Ay	Toplam çalışma	Başarılı	Problemlili	Başarı oranı	Ortalama süre
2026-06	36	35	1	%97,22	142 sn
2026-05	124	122	2	%98,39	138 sn
2026-04	120	119	1	%99,17	141 sn

9.3 Değerlendirme

Bu çalışma sonucunda PostgreSQL 18 üzerinde çalışan dvdrental veritabanı için yedekleme operasyonu script, zamanlama, izleme, uyarı ve doğrulama adımlarını içeren bütünleşik bir otomasyon hattına dönüştürülmüştür. Haftalık tam yedek, günlük değişim odaklı yedek görevi ve saatlik doğrulama kontrolü cron ile düzenli hale getirilmiş; her çalışmanın sonucu dosya sistemi logları ve ops şeması altındaki tablolarla izlenebilir kılınmıştır.

Elde edilen tasarım, manuel komut çalıştırmanın doğurduğu operasyonel belirsizlikleri azaltır. Özellikle exit kodu kontrolü, minimum dosya boyutu doğrulaması, retention temizliği ve e-posta/webhook uyarıları sayesinde başarısız yedeklerin fark edilmeden kalma olasılığı düşürülür. verify.sh betiğinin geçici veritabanına restore yapması ise yedek dosyasının yalnızca oluşturulduğunu değil, geri yüklenebilir olduğunu da kanıtlar.

Sonuç olarak proje, yedekleme tekniklerinin ayrıntılı karşılaştırmasından çok, yedekleme operasyonunun DevOps disipliniyle nasıl tekrarlanabilir, ölçülebilir ve denetlenebilir hale getirileceğini göstermektedir. Bu yaklaşım, dağıtık sistemlerde servis sürekliliği, veri güvenilirliği ve felaket öncesi hazırlık açısından uygulanabilir bir DBA otomasyon modeli sunmaktadır.